
Développement d'applications distribuées

CER | FrontEnd

Lundi 08 juin 2026

TABLE DES MATIÈRES

AAV	2
Étude de cas : Front-end	3
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	4
Livrables	4
Pistes de solutions	5
Plan d'actions	5
Définition : Mots clés	6
JSON	6
Composant	6
React	7
Front-end	7
Wireframe	7
Redux	8
Tailwind CSS	8
useEffect	9
Axios	9
DOM	10
Analyse et résolution : Front-end	11
1. Modélisation et découpage en composants	11
2. Cycle de vie et useEffect	11
3. Chargement asynchrone et gestion des états	12
4. État global et style	13
Synthèse	13
Conclusion	14
Capitalisation	15
Ressources	16

- 5 Appliquer les règles de création des composants
- 6 Construire et manipuler des données en JSON
- 2 Reconnaître les cycles de vie d'un composant
- 3 Réaliser la modélisation d'une application Frontend
- 5 Utiliser un framework Frontend
- 5 Utiliser l'une des librairies de style (Bootstrap, Tailwind, etc.)

ÉTUDE DE CAS : FRONT-END

MOTS CLÉS

- Json
- Composant
- React
- Front-end
- Wireframe
- Redux
- Tailwind CSS
- useEffect
- Axios
- DOM

CONTEXTE

Yanis développe le front-end d'une application avec des composants modulaires via React et Tailwind CSS. Il rencontre des erreurs d'initialisation car les données JSON requises par son composant ne sont pas complètement chargées au moment du rendu de la page.

PROBLÉMATIQUE

Comment gérer efficacement le cycle de vie des composants et le chargement asynchrone des données dans une application React pour éviter les erreurs d'affichage ?

CONTRAINTES

- React
- Tailwind CSS
- JSON

GÉNÉRALISATION

- Front-end
- Composant

LIVRABLES

- Diagramme séquence
- Code source de l'application Front-end

PISTES DE SOLUTIONS

- Se renseigner sur le cycle de vie des composants React et les hooks useEffect
- Se renseigner sur Redux
- Concevoir des composants indépendants et réutilisables

PLAN D' ACTIONS

- DMC
- ER
- Étudier le fonctionnement de React (composants, hooks, états)
- Voir l'architecture de l'application via des wireframes.
- Initialiser un projet React/Next.js.
- Créer les premiers composants
- Installer et configurer Axios
- Implémenter les hooks (useState, useEffect) pour gérer correctement les données et le cycle de vie des composants.
- Gérer la navigation et le routage entre les différentes pages de l'appli

DÉFINITION : MOTS CLÉS

JSON

JSON (*JavaScript Object Notation*) est un **format d'échange de données** léger, textuel et lisible par l'humain, devenu le standard de la communication entre un front-end et une API. Il repose sur deux structures : des **objets** (ensembles de paires "clé": valeur entre accolades) et des **tableaux** (listes ordonnées entre crochets). Exemple de réponse d'API :

```
{ "id": 42, "auteur": "Alice", "tags": ["BD", "manga"], "publie": true }
```

En JavaScript, on **manipule** le JSON avec deux fonctions clés : `JSON.parse()` convertit une chaîne JSON reçue en objet JavaScript exploitable, et `JSON.stringify()` fait l'inverse pour l'envoyer au serveur. C'est l'objet de l'AAV « Construire et manipuler des données en JSON » : les données reçues via Axios sont du JSON, transformées en objets puis affichées dans les composants.

Composant

Un **composant** est la **brique de base, autonome et réutilisable**, d'une interface React. Concrètement, c'est une fonction JavaScript qui reçoit des données en entrée (les **props**) et retourne un morceau d'interface décrit en **JSX** (syntaxe mêlant HTML et JS). Une page se construit en **assemblant** des composants imbriqués, comme des LEGO.

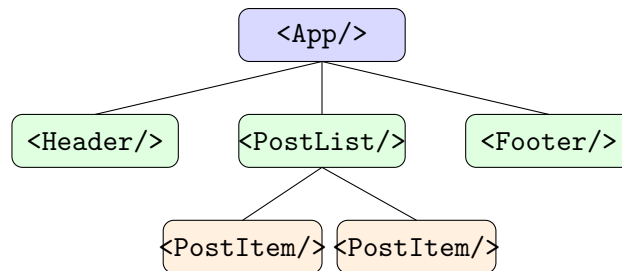


Figure : arbre de composants — une UI React est une hiérarchie de composants réutilisables.

Règles de création (AAV « Appliquer les règles de création des composants ») : nom en **PascalCase** (`PostItem`), **responsabilité unique**, fonction **pure** (mêmes props → même rendu), **props en lecture seule** (jamais modifiées), retour d'un **seul élément racine**, et conception **réutilisable et indépendante**.

React

React est une **bibliothèque JavaScript** front-end (créée par Facebook/Meta) servant à construire des interfaces utilisateur à base de **composants**. C'est le **framework Frontend** retenu pour le projet (AAV « Utiliser un framework Frontend »). Ses principes fondateurs :

- **Déclaratif** : on décrit l'interface en fonction de l'état ; React se charge de mettre à jour l'affichage.
- **Composants** : l'UI est découpée en briques réutilisables.
- **Virtual DOM** : React maintient une copie virtuelle du DOM et ne met à jour que ce qui change réellement, pour de bonnes **performances**.
- **Flux de données unidirectionnel** : les données descendent du parent vers l'enfant via les props.
- **Hooks** : fonctions (`useState`, `useEffect`...) qui ajoutent état et cycle de vie aux composants.

Front-end

Le **front-end** désigne la **partie cliente** d'une application : tout ce qui s'exécute dans le **navigateur** de l'utilisateur et avec quoi il interagit directement (l'interface, l'affichage, l'expérience utilisateur). Il est construit avec **HTML** (structure), **CSS** (style) et **JavaScript** (interactivité), ici via React et Tailwind. Il s'oppose au **back-end** (côté serveur : logique métier, base de données, API) avec lequel il dialogue en échangeant du **JSON** via des requêtes HTTP.

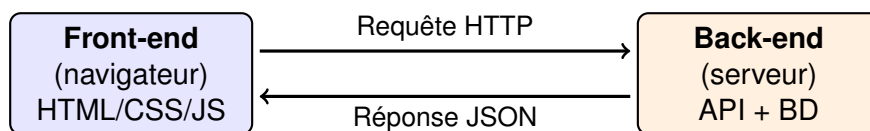


Figure : le front-end (client) dialogue avec le back-end (serveur) via HTTP/JSON.

Wireframe

Un **wireframe** (« maquette filaire ») est un **schéma simplifié, en basse fidélité**, de la structure d'une page ou d'un écran. Sans couleurs ni graphismes définitifs, il sert à positionner les **zones fonctionnelles** (en-tête, navigation, contenu, pied de page) et à valider l'**agencement** avant de coder. C'est un outil central de la **modélisation d'une application Frontend** (AAV) et un préalable au découpage en composants.

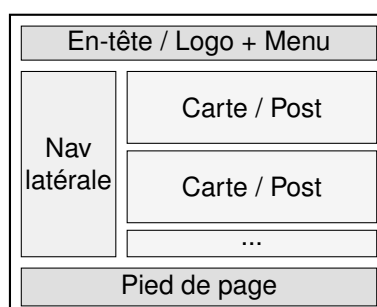


Figure : wireframe d'une page — la structure est définie avant le style et le code.

Redux

Redux est une bibliothèque de **gestion d'état global** pour les applications JavaScript (souvent associée à React). Quand un état doit être **partagé entre de nombreux composants** éloignés (utilisateur connecté, panier, liste de posts), le faire « remonter » via les props devient ingérable. Redux centralise cet état dans un **store unique** et impose un **flux de données unidirectionnel et prévisible** :

- **Store** : source de vérité unique contenant tout l'état.
- **Action** : objet décrivant « ce qui s'est passé » (ex. : { type: 'ADD_POST' }).
- **Reducer** : fonction pure qui calcule le **nouvel état** à partir de l'état courant et de l'action.
- **Dispatch** : déclenche une action depuis l'interface.

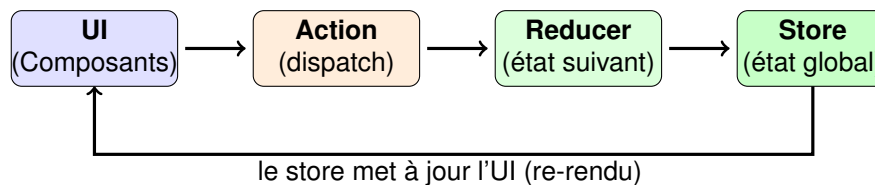


Figure : flux unidirectionnel de Redux — UI → action → reducer → store → UI.

Tailwind CSS

Tailwind CSS est un framework CSS dit « **utility-first** » : au lieu de fournir des composants pré-stylés (comme Bootstrap), il offre une multitude de **classes utilitaires** atomiques que l'on compose directement dans le HTML/JSEX pour styliser élément par élément. Exemple :

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">Envoyer</button>
```

Ici `bg-blue-600` (fond bleu), `text-white` (texte blanc), `px-4 py-2` (marges internes), `rounded` (coins arrondis). **Avantages** : rapidité, cohérence visuelle, design **responsive** simple (md:, lg:), pas de feuilles CSS séparées à maintenir. C'est la **librairie de style** retenue (AAV « Utiliser l'une des librairies de style »). **Différence avec Bootstrap** : Bootstrap impose des composants tout faits (look reconnaissable), Tailwind laisse une liberté totale de design.

useEffect

`useEffect` est le **hook** React qui permet d'exécuter des **effets de bord** (actions extérieures au rendu) : appels à une API, abonnements, minuteurs, manipulation directe du DOM. Il est **central pour gérer le cycle de vie** d'un composant (AAV « Reconnaître les cycles de vie d'un composant ») et répond directement à la **problématique du projet** (chargement asynchrone des données). Son comportement dépend du **tableau de dépendances** :

- `useEffect(fn, [])` : s'exécute **une seule fois au montage** — idéal pour charger les données initiales.
- `useEffect(fn, [x])` : se ré-exécute à chaque changement de `x` (mise à jour).
- **Fonction de nettoyage** (`return`) : exécutée au démontage, pour annuler abonnements/requêtes et éviter les fuites.



Figure : les trois phases du cycle de vie pilotées par `useEffect`. Dans le projet, on charge les données dans un `useEffect([])` et on affiche un état de chargement tant que le JSON n'est pas arrivé — ce qui élimine les erreurs d'initialisation décrites dans le contexte.

Axios

Axios est une bibliothèque cliente **HTTP basée sur les promesses** (*Promise*) permettant au front-end d'effectuer des **requêtes asynchrones** vers une API REST (GET, POST, PUT, DELETE) et d'en récupérer les données. Atouts par rapport au `fetch` natif : conversion **automatique en JSON**, gestion simplifiée des erreurs, en-têtes et intercepteurs, délais d'attente. Couplé à `useEffect`, il constitue le mécanisme de **chargement asynchrone** des données :

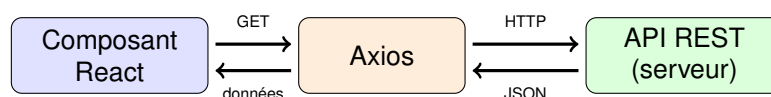


Figure : Axios fait le pont asynchrone entre le composant React et l'API REST.

DOM

Le **DOM** (*Document Object Model*) est la **représentation en mémoire, sous forme d'arbre**, de la page HTML telle qu'interprétée par le navigateur. Chaque balise HTML y devient un **nœud** manipulable par JavaScript (créer, modifier, supprimer des éléments, réagir aux événements). C'est via le DOM que l'interface est réellement affichée et mise à jour.

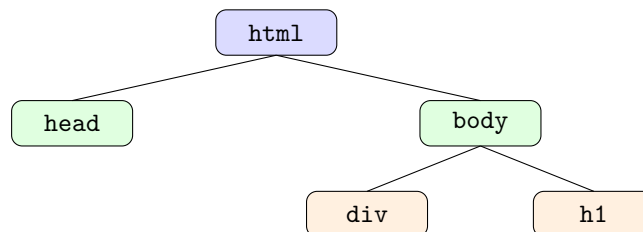


Figure : le DOM, arbre des nœuds de la page. React n'agit pas directement sur ce DOM réel (coûteux) : il passe par un **Virtual DOM**, en calcule les différences, et ne met à jour que les nœuds modifiés — d'où ses bonnes **performances**.

AJAX (Asynchronous JavaScript and XML)

AJAX (*Asynchronous JavaScript and XML*) est une technique de développement web qui permet à une page d'échanger des données avec un serveur en arrière-plan, sans avoir à recharger toute la page.

Concrètement, au lieu de recharger entièrement une page à chaque action de l'utilisateur, le navigateur envoie des requêtes au serveur de façon asynchrone (en tâche de fond) et met à jour seulement les parties concernées de la page. L'utilisateur peut continuer à interagir avec la page pendant que la requête est traitée.

- **Asynchrone** : les requêtes n'interrompent pas l'utilisation de la page. Le code n'attend pas la réponse pour continuer.
- **Les briques techniques** : historiquement l'objet XMLHttpRequest, aujourd'hui plus souvent l'API `fetch()` ou des bibliothèques comme Axios. Malgré le nom, on échange aujourd'hui surtout du JSON plutôt que du XML.
- **Cas d'usage typiques** : suggestions de recherche en temps réel (autocomplétion), validation de formulaire sans rechargement, fil d'actualité qui se met à jour, chargement de contenu au défilement, etc.

Exemple avec fetch

```
fetch('/api/donnees')
  .then(response => response.json())
  .then(data => {
    // mise à jour du DOM avec les données reçues
    console.log(data);
  })
  .catch(error => console.error(error));
```

ANALYSE ET RÉOLUTION : FRONT-END

Rappel de la problématique : « Comment gérer efficacement le cycle de vie des composants et le chargement asynchrone des données dans une application React pour éviter les erreurs d’affichage ? »

Pour y répondre, nous traitons successivement les quatre facettes du problème : **(1) la modélisation** de l’interface et son découpage en composants, **(2) le cycle de vie** d’un composant et le hook `useEffect`, **(3) le chargement asynchrone** des données (Axios) et la **gestion des états** qui élimine les erreurs d’affichage, **(4) l’état global** (Redux) et le **style** (Tailwind). Une synthèse argumentée clôt l’analyse.

1. MODÉLISATION ET DÉCOUPAGE EN COMPOSANTS

La conception débute **avant le code** par un **wireframe** qui fixe la structure des écrans (en-tête, navigation, contenu, pied de page). Ce découpage se traduit ensuite en un **arbre de composants** où chaque zone fonctionnelle devient une brique autonome (`<Header/>`, `<PostList/>`, `<PostItem/>`, `<Footer/>`). Les **règles de création** sont appliquées : nom en **PascalCase**, **responsabilité unique**, fonction **pure**, **props en lecture seule** et **réutilisabilité**. Un composant feuille typique ne fait que recevoir des props et retourner du JSX :

```
// PostItem.jsx -- composant pur, réutilisable, props en lecture seule
function PostItem({ auteur, contenu }) {
  return (
    <li className="p-4 rounded shadow bg-white">
      <strong>{auteur}</strong> : {contenu}
    </li>
  );
}
```

2. CYCLE DE VIE ET HOOK USEEFFECT

Un composant traverse trois phases : le **montage** (première insertion dans le DOM), les **mise à jour** (re-rendus déclenchés par un changement d’état ou de props) et le **démontage** (retrait du DOM). L’erreur décrite dans le contexte vient d’un **mauvais placement de la logique de chargement** : si l’on tente d’exploiter les données pendant le rendu, le composant s’exécute **avant** l’arrivée du JSON. La solution consiste à déporter l’appel réseau dans un `useEffect` déclenché **au montage** (tableau de dépendances vide `[]`) : le rendu n’attend pas le réseau, et c’est l’arrivée des données qui provoque un re-rendu propre.

3. CHARGEMENT ASYNCHRONE ET GESTION DES ÉTATS

Le cœur de la résolution combine **Axios** (requête HTTP asynchrone, JSON déjà parsé) avec **trois états** maintenus par `useState` : les **données**, un indicateur de **chargement** et une éventuelle **erreur**. L'appel est lancé au montage et une **fonction de nettoyage** annule la requête au démontage (`AbortController`), évitant les fuites et les mises à jour sur composant démonté.

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function PostList() {
  const [posts, setPosts] = useState([]); // données (JSON -> objets)
  const [chargement, setLoad] = useState(true); // état de chargement
  const [erreur, setErreur] = useState(null); // état d'erreur

  useEffect(() => {
    const controleur = new AbortController(); // annulation au démontage

    axios.get('/api/posts', { signal: controleur.signal })
      .then((reponse) => setPosts(reponse.data)) // JSON déjà parsé par Axios
      .catch((err) => { if (!axios.isCancel(err)) setErreur(err); })
      .finally(() => setLoad(false));

    return () => controleur.abort(); // nettoyage (démontage)
  }, []); // [] -> une fois, au montage
```

Le **rendu conditionnel** garantit ensuite que l'on n'affiche **jamais** la liste tant que les données ne sont pas disponibles : c'est précisément ce qui **supprime les erreurs d'affichage** de la problématique.

```
if (chargement) return <p className="text-gray-500">Chargement...</p>;
if (erreur) return <p className="text-red-600">Erreur de chargement.</p>;

return (
  <ul className="space-y-2">
    {posts.map((post) => (
      <PostItem key={post.id} auteur={post.auteur} contenu={post.contenu} />
    ))}
  </ul>
);
```

Tant que `chargement` est vrai, l'interface affiche un indicateur ; en cas d'échec, un message d'erreur ; ce n'est qu'**une fois le JSON arrivé** que la liste est rendue. Le composant est ainsi **robuste à l'asynchronisme**, ce qui répond directement au contexte (rendu déclenché avant la disponibilité des données).

4. ÉTAT GLOBAL (REDUX) ET STYLE (TAILWIND)

Lorsqu'une donnée doit être **partagée entre de nombreux composants éloignés** (utilisateur connecté, liste de posts mutualisée), la faire transiter par les props devient ingérable. **Redux** centralise alors cet état dans un **store unique** au flux unidirectionnel et prévisible (*action* → *reducer* → *store* → re-rendu de l'UI), ce qui rend l'évolution de l'application plus lisible et testable.

Côté présentation, **Tailwind CSS** (*utility-first*) applique le style directement dans le JSX via des classes atomiques (`p-4`, `rounded`, `bg-white`, `text-red-600...`), comme dans les extraits ci-dessus. On obtient un rendu **cohérent et responsive** sans feuille CSS séparée à maintenir.

SYNTHÈSE

La réponse à la problématique tient en une chaîne cohérente : **modéliser** (wireframe → arbre de composants), **isoler l'asynchrone** dans un `useEffect` de montage, **charger** via `Axios` en pilotant trois états (données / chargement / erreur), **rendre conditionnellement** pour ne jamais afficher de données absentes, et **nettoyer** au démontage. Redux et Tailwind complètent la pile pour l'état partagé et le style. Cette démarche élimine les erreurs d'initialisation et constitue un **patron reproductible** pour toute page consommant une API.

CONCLUSION

En réponse à la problématique, l'analyse a montré que gérer efficacement le cycle de vie des composants et le chargement asynchrone des données ne relève pas d'une astuce isolée, mais d'une **chaîne de décisions cohérentes**. Tout part de la **modélisation** : un wireframe fixe la structure des écrans, traduite en un **arbre de composants** modulaires, purs et réutilisables (PascalCase, responsabilité unique, props en lecture seule).

Le cœur de la résolution est le **placement correct de la logique asynchrone** : l'appel réseau est déporté dans un `useEffect` déclenché au **montage** (`[]`), de sorte que le rendu n'attend jamais le réseau. **Axios** récupère le **JSON** et alimente trois états (données, chargement, erreur) qui pilotent un **rendu conditionnel** : on n'affiche la liste qu'une fois les données présentes. C'est ce mécanisme qui **supprime les erreurs d'affichage** décrites dans le contexte. Une **fonction de nettoyage** (`AbortController`) annule la requête au démontage et évite les fuites mémoire.

Pour l'état partagé entre composants éloignés, **Redux** centralise les données dans un store unique au flux unidirectionnel et prévisible, tandis que **Tailwind CSS** assure un style cohérent et responsive directement dans le JSX. Au final, la pile **React + Axios + Redux + Tailwind** satisfait l'ensemble des AAV visés et constitue un **patron reproductible** pour le développement front-end des futures applications de l'entreprise.

CAPITALISATION

AAV	Commentaire	Statut
Appliquer les règles de création des composants	Règles détaillées (PascalCase, responsabilité unique, fonction pure, props en lecture seule, racine unique) et mises en œuvre dans le composant <code>PostItem</code> de l'analyse.	Acquis
Construire et manipuler des données en JSON	Structure du JSON (objets/tableaux) expliquée ; réponse d'API récupérée via <code>Axios</code> (<code>reponse.data</code>) puis parcourue (<code>.map</code>) pour alimenter les composants.	Acquis
Reconnaître les cycles de vie d'un composant	Phases montage / mise à jour / démontage schématisées et pilotées par <code>useEffect</code> (dépendances <code>[]</code> , nettoyage par <code>AbortController</code>) dans le code de résolution.	Acquis
Réaliser la modélisation d'une application Frontend	Modélisation par wireframes (structure des écrans) puis découpage en arbre de composants, repris comme point de départ de l'analyse.	Acquis
Utiliser un framework Frontend	React expliqué (composants, déclaratif, Virtual DOM, flux unidirectionnel, hooks) et appliqué concrètement (<code>useState/useEffect</code> , rendu conditionnel) pour résoudre la problématique.	Acquis
Utiliser l'une des bibliothèques de style (Bootstrap, Tailwind, etc.)	Tailwind CSS (utility-first) présenté avec exemple de classes, comparé à Bootstrap et utilisé directement dans le JSX des extraits de l'analyse.	Acquis

RESSOURCES

- **Documentation React** : <https://react.dev/>
- **Hook useEffect (cycle de vie)** : <https://react.dev/reference/react/useEffect>
- **Documentation Tailwind CSS** : <https://tailwindcss.com/docs>
- **Documentation Redux / Redux Toolkit** : <https://redux.js.org/>
- **Documentation Axios** : <https://axios-http.com/fr/docs/intro>
- **JSON (MDN)** : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/JSON
- **DOM (MDN)** : https://developer.mozilla.org/fr/docs/Web/API/Document_Object_Model
- **Next.js (framework React)** : <https://nextjs.org/docs>